

Intelligent Robotics Assignment 1 Report

Group 11

Particle Cloud Initialisation

Design Choices

To initialise the particle cloud, new poses are generated by adding random offsets to the position and orientation of the initial pose estimate until the desired number of particles have been created. The offsets for the X and Y coordinates and for the heading are each sampled from a normal distribution with a specified variance. There is a trade-off to consider when choosing the variances. If you use smaller values, the initial cloud will be closer to the given estimate. If the estimate is accurate this will allow the robot to quickly localise, but with a poor estimate the robot will not be able to localise correctly. If you use larger values the accuracy of the initial estimate has less of an effect on the robot's ability to localise, as the particles will have lots of variation regardless of the estimate. This is helpful when the given estimate is poor, but risks ignoring an accurate initial estimate and causing the robot to spend more time localising than it would otherwise need to.

Experimental Results

In our earliest implementation, the cloud update was very simple and wasn't likely to correct itself if the cloud converged at an incorrect pose. Because of this, we initially settled on large values for the variances of the initial cloud to maximise the robot's chance of localising correctly soon after initialisation. For both the X and Y offsets we used a variance of 10, and for the heading (in radians) we used a variance of 6. This gave the robot a chance of localising correctly if the estimated pose was in vaguely the right area, although the robot would still usually localise to the wrong pose. In our final implementation, the cloud update is much more sophisticated and can correct the particle cloud when the pose is likely to be incorrect. Because of this, the spread of the initial cloud became much less important because the cloud was able to spread itself out again as needed. As a result, we reduced the variances for the initial cloud to 2 for the X and Y offsets and 1 for the heading offset.

Particle Cloud Update: Roulette-Wheel Sampling

Design Choices

In our initial implementation we chose to resample using a roulette-wheel algorithm due to its simplicity making it quick to implement. We later replaced it with systematic sampling (stochastic universal sampling) once we had that written and tested. The roulette-wheel implementation uses a binary search over an array of the cumulative weights.

Experimental Results

We tested the roulette-wheel sampling in isolation before using it for the particle filter, to ensure its correctness. It was important that it worked properly so that we could compare its behaviour to our systematic sampling implementation later on. Our tests involved providing the "random" target value manually and then checking that the search would return the correct index. When we were satisfied that the implementation was correct, we applied it to the particle filtering.

Particle Cloud Update: Systematic Sampling

Design Choices

Whilst our roulette-wheel sampling worked well, it had a time complexity of $O(n \log n)$. We therefore wanted to update our sampling method so that it used systematic sampling instead. Systematic sampling chooses a random value within the total weights, and then samples at regular intervals throughout the particles, checking which particle the new sample falls in. This only requires 3 for loops (in our implementations), none of which are

nested within each other. This means our worst case time complexity is $3n$, leading to the time complexity to be $O(n)$.

Experimental Results

Similar to roulette sampling, we tested this method in isolation, using a set array of cumulative weights. We would just keep running the method with a different random value and check that the particles are consistently being selected proportionally to their weights. After testing both roulette and systematic with no other changes to the code, there is no visual difference in how the sampling works concerning effectiveness, which is intended. In regards to performance, this change from $O(n \log n)$ to $O(n)$ won't be noticeable with the amount of particles we are using, but if we were dealing with hundreds or thousands, the systematic sampling may be noticeably more efficient.

Particle Cloud Update: Adding Random Particles

Design Choices

To tackle the robot's inability to correct itself when the particle cloud converges in the wrong place, we decided that some randomly-placed particles should have a chance of being sampled in each step. These particles are generated by adding normally-distributed offsets to the position and orientation of the current pose estimate, similar to the particle cloud initialisation (although with separate variance settings). It is worth noting that the effectiveness of the pose estimation impacts the effectiveness of the particle cloud updating because of this. Our first design for this idea involved adding a fixed number of these random particles after the resampling was complete, so that there were always some random particles in the cloud.

Experimental Results

In practice the random particles did give the robot a chance to correct itself when its estimate was wrong, and we observed this numerous times in testing. However, there were some problems with the behaviour. Firstly, configuring the spread of the random particles was very challenging: too little spread and the robot could only recover from small errors, too much spread and there aren't enough random particles to cover the area in a reasonable time. Because the number of random particles can't exceed the number of regular particles we can't just add more random particles. Second, the random particles would sometimes cause the cloud to 'correct' itself from a correct pose to an incorrect one, particularly where there are multiple environments of roughly the same shape or with some symmetry. Thirdly, the random particles caused the pose estimate to become unstable, although this was because at this point our pose estimation algorithm simply took the average pose across all of the particles. With the more advanced pose estimation algorithms described below this problem became much less prominent.

Changes to the Design

To address the first two issues, we adjusted the design so that the random particles are generated before the resampling, instead of after. They are added to the set of particles being sampled from, and have their weights calculated by the sensor model as with the regular particles. This has two advantages, the first being that the number of random particles is no longer limited by the number of regular particles. This allows us to increase the spread of the random particles arbitrarily if needed and still search most of the area in reasonable time, addressing the first issue. The second advantage is that random particles which are not likely poses will be given low weights and so are less likely to be resampled. This partially addresses the second issue as the random particles are no longer guaranteed to be in the final resampled set of particles. It also partially addresses the third issue for the same reason.

Experimental Results

In practice the changes did seem to help as we expected: the random particles could now cover a larger area, allowing the robot to 'correct' itself over larger distances; the low-weight random particles were less likely to be resampled, so the robot would falsely 'correct' itself less often; and the estimated pose would be slightly more stable as there wouldn't always be many random particles being resampled. This wasn't a complete fix, however. When the robot was correctly localised there would still be some random particles making it through the resampling, making the pose estimate less stable and giving a slight chance of the cloud switching away from the correct pose to another (albeit likely) pose. To fix this, we tried an adaptive approach.

Particle Cloud Update: Adaptive Approach

Design Choices

The aim of this feature is to adjust the number of randomly added particles such that more are generated when the existing particles do not give a 'confident' estimation of the true pose. We achieve this using a "likelihood-based approach", i.e. we keep generating particles until the sum of all the weights reaches some predetermined threshold, or until some maximum number of particles has been reached. This has the effect of ensuring that the robot has a minimum amount of 'confidence' in the estimated poses before it resamples them. It also means that if the robot is already confident in the existing particles, no random ones need to be generated. Note that the number of samples taken doesn't change, only the number of particles being sampled from.

An additional feature of our design is that each time a random particle is added, the variance used to generate it is multiplied by a small factor (we settled on 1.05). The intention behind this is that if lots of particles have been added and the sum of the weights is still too low, we should add particles further away as the robot is likely to be a long way from its estimate.

Experimental Results

This approach has worked well in our testing, although it took a lot of fine-tuning to get it to work well. We found that the maximum number of random particles can be quite high without any problems, as typically only a few are generated if any, and it allows the particles to become very spread out if the estimate is a long way from the true pose. To determine what to use as the weight sum threshold, we looked at what the weight values were when the robot is correctly localised compared to when it is not. We found that the weights would have values of roughly 0-8 when the particle cloud gave a poor estimate, compared to values of roughly 9-15 when the particles had converged on the correct location. We found that 30 (regular, non-random) particles worked well, so we expected that a threshold of roughly $30 \times 8 = 240$ would be appropriate.

The trade-off to consider when setting the weight sum threshold is this: if it is too high then the cloud will likely never converge, as it will always need to add lots of random particles, whereas if it is too low then the cloud is more likely to converge on an incorrect pose estimate. After a lot of experimentation we found that using 200 as the threshold worked best, which is reasonably close to our prediction. With these settings, and rudimentary pose estimation, the robot was able to localise in most situations, although still sometimes it would settle on an incorrect estimate for a while before correcting itself. The incorrect estimates were typically caused by similarities and symmetries between areas of the map however, and were usually reasonable guesses because of this.

Particle Cloud Update: Exploiting Symmetry

Design Choices

One last improvement we tried to add to the random particles came from two observations. The first is that often when the robot was correcting its pose estimate, a particle would be generated by chance in the correct position but with the wrong orientation. The second is

that the robot's incorrect pose estimates were often caused by symmetry in the map, where some areas had very similar physical characteristics to others. The incorrect pose estimates would typically be either 90 or 180 degrees offset from the true estimate. To try and fix this, we modified the random particle generation such that after the orientation offset has been sampled from its normal distribution, it is rotated by 0, 90, 180 or 270 degrees with equal probability. This has the effect of making the particles most likely to be offset by 90 or 180 degrees from the current pose estimate, which means the random particles' orientations should on average be more accurate to the true pose.

Experimental Results

In the testing we performed in the time available we haven't seen convincing evidence that this feature helps the robot to localise, although we also haven't seen convincing evidence that makes the localisation less effective.

Pose Estimation: Averaging

Design Choices

Our first attempt at pose estimation was to simply average all of the poses in the particle cloud. This was sufficient at first, but once we introduced random particles the estimate became very unstable. To try and minimise this we adjusted the algorithm such that any particles whose X or Y coordinate was further away from the mean than the standard deviation (multiplied by a constant, which we kept at 1) was ignored when taking the average. This was intended to prevent situations where most of the cloud was converged, but a small number of random particles were affecting the estimate too much.

Experimental Results

We found that this rudimentary method of pose estimation had the desired effect of stabilising the average, in spite of the random particles. When combined with all of the other features listed above, this type of pose estimation was sufficient for the robot to localise reasonably well, although it would still sometimes settle on incorrect estimates for a while before correcting. To try and improve this, we tried some more sophisticated methods of pose estimation.

Pose Estimation: kNN Clustering

Design Choices

Our second attempt at pose estimation was to use kNN clustering. Our approach in implementing kNN was to find an average pose by averaging all the poses in the particle cloud. Then, calculate the distance between all the poses in the particle cloud and the average pose. After that, we choose the best neighbours to the average pose and use them to get the best pose estimate.

Experimental Results

This algorithm was able to localise the robot well most of the time. This approach worked well in localising the robot when used with all the features that we implemented. Although this simple algorithm worked well most of the time, it used to fail to localise the robot when the robot is lost. The robot had to go near to the estimated pose till it can localise itself again, thus this approach wasn't good enough.

Pose Estimation: K-means Clustering

Design Choices

We decided to implement K-means Clustering in our pose estimation function both to improve the pose estimate for when the robot is confident in its location but also when the robot is considering multiple possible locations. The clustering for confident locations improves the pose estimate as it gives an average location of the most likely particles, easily disregarding particles that are somewhat nearby but suggest a different pose.

When considering the kidnapped robot problem, this form of clustering is particularly useful. The particle update cloud frequently considers multiple locations around the map when lost, typically taking the form of several clusters of particles. Without this clustering these frequently fail to become the dominant location hypothesis as the random particles which are needed to give more weight to these locations are placed based on a distribution centred on the pose estimate. When the k-means clustering is enabled the pose estimate is placed upon one of these clusters meaning the randomly added particles are more likely to add to the number of particles suggesting that location giving it an increased chance of becoming the dominant location estimate.

The number of clusters for the particles to be sorted into was determined by viewing the average number of naturally occurring when the robot was lost being 3.

An important design choice was the use of numpy arrays to store the assignments of the particles to clusters. This improved the performance compared to built in python lists allowing for the total number of particles to be increased. The time complexity of this function was kept to $O(N)$ where n is the number of particles. The actual complexity is very dependent on the initialisation of the clusters as a loop through every particle occurs every single time any of the cluster locations is updated which occurs many times. This is somewhat mitigated by using the numpy allclose method to ignore very small changes in the cluster movement but is still computationally intensive. Limiting the number of times the cluster could update was considered but ultimately rejected as if it was taking too long the method would be called again anyway and setting the limit too low could result in badly placed clusters.

Experimental Results

As described above the chosen number of clusters was 3. Other values were tried with the earlier version of the entire program to some success but 3 always performed best. With the latest version of the particle update having 3 clusters was the only value to give reasonable pose estimations. This is likely due to its increased number of particles and natural tendency to form 3 clusters.

Another experiment was how to best find the cluster to put the pose on. Initially the best cluster was found purely by the number of particles assigned to it. This worked quite well but we thought using the variance within a cluster could give a better result. This consistently returned bad poses and was dropped in favour of keeping the system of choosing the cluster with most assigned particles.

Conclusion and Comparison to AMCL Implementation

Across the various different methods of localisation which we've tried, enough of them have been successful that we have assembled them into a final model. For the initialisation of the cloud we've stuck with our initial Gaussian-based approach. For the update step we've combined systematic sampling and an adaptive number of additional random particles, allowing the robot to correct itself even when its estimate is far from the true pose. For the pose estimation we've employed K-means clustering to entertain multiple estimates until one becomes most likely, basing estimations on sensible groupings of particles.

Compared to the AMCL implementation, our approach performs better in some areas and worse in others. AMCL is good at maintaining a confident estimate of the robot's pose over long distances, whereas our approach might re-localise to an incorrect location if the weights drop too low. Given a good initial pose estimate, both implementations are able to follow the robot around the map, but AMCL will not correct itself if given a poor initial estimate, unlike our implementation. This can cause AMCL to give invalid poses (i.e. poses which intersect or are outside of the map), which is much less common in our approach. Similarly, AMCL does not cope well with the kidnapped robot problem compared to our implementation, as its particle cloud does not spread out nearly as far. However, our implementation may take a long time to settle on a stable estimate in this situation, whereas AMCL's estimate will fluctuate much less. Overall we believe our implementation is a strong attempt at robot localisation.